

Machine Learning Library From Scratch

High-Performance Classical ML Systems Project

Project Documentation

Last updated: 2026-05-28

1. Project Overview

This project is an end-to-end implementation of a classical machine learning library built entirely from scratch, with a strong focus on:

- ? Understanding machine learning algorithms at a systems level.
- ? Reproducing the architecture philosophy of mature ML frameworks.
- ? Building high-performance CPU implementations.
- ? Developing low-level optimization skills.
- ? Studying the intersection of machine learning, numerical computing, and systems engineering.

The project originally began as a high-performance implementation of Linear Regression and Logistic Regression without using ML libraries. It has now expanded into a complete classical machine learning library covering the core algorithms and techniques from the first major section of Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurelien Geron.

The goal is no longer just implementing individual algorithms, but building a cohesive, extensible, optimized machine learning ecosystem from scratch.

The long-term vision is to create a lightweight CPU-oriented ML framework with:

- ? A clean scikit-learn-inspired API.
- ? Efficient numerical kernels.
- ? Modular architecture.
- ? Reusable optimization infrastructure.
- ? High-performance implementations in C.
- ? Full educational transparency.

This project is simultaneously:

- ? A machine learning project.
- ? A systems programming project.

- ? A numerical computing project.
- ? A performance engineering project.
- ? A software architecture project.

2. Core Philosophy

The project is built around five major principles.

2.1 Learn by Rebuilding

Every algorithm should be implemented from first principles.

No ML libraries are used for the core implementation:

- ? No scikit-learn.
- ? No PyTorch.
- ? No TensorFlow.
- ? No XGBoost.
- ? No LightGBM.
- ? No CatBoost.
- ? No NumPy-based algorithm implementations.

The purpose is not merely using algorithms, but understanding:

- ? Their mathematics.
- ? Their optimization behavior.
- ? Their memory access patterns.
- ? Their computational bottlenecks.
- ? Their numerical stability challenges.
- ? Their scaling properties.

2.2 Systems-Level Thinking

The library should mirror real-world ML systems design:

- ? High-level API in Python.
- ? Performance-critical kernels in C.
- ? Efficient memory layouts.
- ? Cache-aware data access.

- ? SIMD vectorization.
- ? Multithreading.
- ? Reduced Python overhead.

The project intentionally bridges the gap between:

- ? Machine learning engineering.
- ? Systems programming.
- ? High-performance computing.

2.3 Incremental Optimization

Every component should evolve through optimization stages:

1. Correctness-first implementation.
2. Profiling.
3. Bottleneck identification.
4. Algorithmic optimization.
5. Memory optimization.
6. Vectorization.
7. Parallelization.
8. Benchmark validation.

The emphasis is on measurable and reproducible performance improvements.

2.4 Reusable Infrastructure

Algorithms should share reusable internal systems whenever possible.

Examples:

- ? Shared data containers.
- ? Shared metrics.
- ? Shared optimizers.
- ? Shared preprocessing utilities.
- ? Shared tree structures.
- ? Shared ensemble interfaces.
- ? Shared validation pipelines.
- ? Shared benchmark infrastructure.

The project should gradually evolve from isolated scripts into a real software library.

2.5 Transparency Over Abstraction

The implementation should remain understandable.

Even when highly optimized, the codebase should preserve:

- ? Clear architecture.
- ? Traceable data flow.
- ? Documented optimizations.
- ? Reproducible experiments.
- ? Explainable implementation decisions.

3. Scope of the Project

The project now includes the implementation of most major classical machine learning algorithms covered in the foundational section of Geron's handbook.

3.1 Linear Models

Regression:

- ? Linear Regression
- ? Polynomial Regression
- ? Ridge Regression
- ? Lasso Regression
- ? Elastic Net
- ? SGD Regression

Classification:

- ? Logistic Regression
- ? Softmax Regression
- ? SGD Classifier

Key learning goals:

- ? Gradient descent variants.
- ? Convex optimization.

- ? Regularization.
- ? Numerical stability.
- ? Feature engineering.
- ? Bias-variance tradeoff.

3.2 Tree-Based Models

- ? Decision Trees for classification and regression
- ? CART-style splitting
- ? Entropy and Gini impurity
- ? Tree pruning
- ? Random Forests
- ? Extremely Randomized Trees

Key learning goals:

- ? Recursive partitioning.
- ? Information theory.
- ? Greedy optimization.
- ? Tree traversal.
- ? Ensemble variance reduction.

3.3 Ensemble Learning

- ? Bagging
- ? Pasting
- ? Random Forests
- ? AdaBoost
- ? Gradient Boosting
- ? Voting Classifiers
- ? Stacking as an optional advanced extension

Key learning goals:

- ? Weak learners.
- ? Bias reduction.
- ? Variance reduction.
- ? Sequential learning.

- ? Residual fitting.
- ? Ensemble architecture.

3.4 Support Vector Machines

- ? Linear SVM
- ? Soft-margin classification
- ? Kernel methods
- ? Polynomial kernel
- ? RBF kernel

Key learning goals:

- ? Margin maximization.
- ? Dual optimization.
- ? Kernel tricks.
- ? High-dimensional geometry.

3.5 Unsupervised Learning

Clustering:

- ? K-Means
- ? Mini-Batch K-Means
- ? DBSCAN
- ? Gaussian Mixture Models as an optional advanced target

Dimensionality reduction:

- ? PCA
- ? Incremental PCA
- ? Randomized PCA

Key learning goals:

- ? Distance metrics.
- ? Latent structure discovery.
- ? Eigen decomposition.
- ? Numerical linear algebra.

3.6 Utility Components

Preprocessing:

- ? StandardScaler
- ? MinMaxScaler
- ? OneHotEncoder
- ? Train/Test Split
- ? Feature normalization
- ? Polynomial feature expansion

Metrics:

- ? MSE
- ? RMSE
- ? MAE
- ? R2
- ? Accuracy
- ? Precision
- ? Recall
- ? F1 Score
- ? Confusion Matrix

Validation:

- ? Cross-validation
- ? Grid search as a future extension
- ? Random search as a future extension

4. Constraints and Rules

Allowed:

- ? Python Standard Library
- ? C Standard Library
- ? Compiler toolchains such as GCC and Clang
- ? OS tooling such as perf, valgrind, gprof, and sanitizers

Python Standard Library examples:

- ? ctypes
- ? array
- ? math
- ? csv
- ? random
- ? time
- ? statistics
- ? threading
- ? multiprocessing

Optional comparison-only dependencies:

- ? scikit-learn
- ? NumPy
- ? pandas

These may be used only for benchmarking or correctness comparison. They must never power the core implementation.

Not allowed in core implementations:

- ? scikit-learn algorithms
- ? NumPy-based ML implementations
- ? SciPy optimizers
- ? PyTorch
- ? TensorFlow
- ? XGBoost
- ? LightGBM
- ? CatBoost
- ? External BLAS/LAPACK for the primary implementation

The objective is implementation ownership.

5. System Architecture

The library is designed as a layered architecture.

5.1 High-Level Structure

The implementation is split into:

Python API layer, responsible for:

- ? User-facing APIs
- ? fit/predict/score interfaces
- ? Dataset loading
- ? Validation
- ? Benchmarking
- ? Experiment orchestration
- ? Testing
- ? Pipeline composition

Native C kernel layer, responsible for:

- ? Matrix operations
- ? Vector operations
- ? Gradient computation
- ? Tree split evaluation
- ? Histogram accumulation
- ? Distance calculations
- ? Reduction operations
- ? SIMD execution
- ? Parallel loops

Python acts as the conductor. C acts as the engine room.

5.2 Data Representation

Performance depends heavily on memory layout.

Design goals:

- ? Contiguous memory.
- ? Cache-friendly traversal.
- ? Reduced allocations.
- ? Minimal copying.
- ? SIMD-friendly alignment.

Planned layout:

- ? Row-major dense arrays.
- ? float64 for correctness.
- ? Optional float32 optimization path.
- ? array('d') and array('f') in Python.

Future extensions may include:

- ? Sparse matrix representations.
- ? Memory pools.
- ? Arena allocators.
- ? Aligned allocators.

5.3 API Philosophy

The library aims for a scikit-learn-inspired interface.

Example:

```
model = LinearRegression()  
model.fit(X_train, y_train)  
preds = model.predict(X_test)  
score = model.score(X_test, y_test)
```

Consistency across algorithms is a major design objective.

6. Numerical and Optimization Goals

6.1 Correctness First

Every implementation should first pass:

- ? Unit tests.
- ? Numerical checks.
- ? Convergence checks.
- ? Cross-validation against reference outputs.

Only after correctness is validated should optimization begin.

6.2 Optimization Objectives

The project aims to progressively approach the efficiency of industrial libraries.

Optimization areas include:

- ? Loop fusion.
- ? Cache locality.
- ? Branch reduction.
- ? SIMD vectorization.
- ? OpenMP multithreading.
- ? Reduced interpreter overhead.
- ? Memory reuse.
- ? Batch processing.
- ? Kernel fusion.

6.3 CPU-Oriented Performance Engineering

The library is intentionally CPU-focused.

Major learning areas:

- ? Cache hierarchies.
- ? Memory bandwidth.
- ? SIMD instruction sets.
- ? Thread scheduling.
- ? False sharing.
- ? Reduction strategies.
- ? Profiling methodology.

Potential SIMD targets:

- ? AVX2
- ? AVX-512
- ? ARM NEON

7. Algorithm Development Strategy

Every algorithm follows a staged implementation roadmap.

Stage 1: Mathematical Prototype

- ? Simple implementation.
- ? Readability prioritized.
- ? Easy debugging.
- ? Small-scale testing.

Stage 2: Correctness Validation

- ? Unit tests.
- ? Numerical gradient checks.
- ? Cross-checks with reference implementations.
- ? Synthetic dataset validation.

Stage 3: Profiling

Identify:

- ? Hot loops.
- ? Memory bottlenecks.
- ? Allocation overhead.
- ? Branch-heavy regions.

Stage 4: Native Kernel Extraction

Move compute-heavy logic into C.

Typical targets:

- ? Dot products.
- ? Distance calculations.
- ? Histogram accumulation.
- ? Split scoring.
- ? Residual computation.

Stage 5: Vectorization

- ? Compiler auto-vectorization.
- ? SIMD intrinsics.
- ? Data alignment.
- ? Loop restructuring.

Stage 6: Parallelization

- ? OpenMP.
- ? Thread-safe reductions.
- ? Work partitioning.
- ? Batch parallelism.

8. Example Kernel Concepts

8.1 Linear Regression Epoch Kernel

```
int linreg_epoch_f64(  
    const double* x,  
    const double* y,  
    double* w,  
    double* b,  
    size_t n,  
    size_t d,  
    double lr  
);
```

Responsibilities:

- ? Prediction
- ? Residual computation
- ? Gradient accumulation
- ? Parameter updates

8.2 Decision Tree Split Kernel

Potential responsibilities:

- ? Histogram construction
- ? Gini/entropy computation
- ? Best split search
- ? Partition generation

8.3 K-Means Distance Kernel

Responsibilities:

- ? Distance computation

- ? Cluster assignment
- ? Partial reduction
- ? SIMD acceleration

9. Benchmarking Methodology

Benchmarking must be reproducible and fair.

9.1 Datasets

Synthetic datasets with controlled dimensions:

- ? n in $\{10k, 100k, 1M\}$
- ? d in $\{10, 50, 200, 1000\}$

Real datasets:

- ? California Housing
- ? Iris
- ? MNIST subsets
- ? Wine dataset
- ? Breast Cancer dataset

Used only for:

- ? Validation
- ? Real-world behavior
- ? Comparative benchmarking

9.2 Metrics

Performance:

- ? Epoch time
- ? Total fit time
- ? Prediction latency
- ? Memory usage
- ? Throughput

Quality:

- ? MSE
- ? RMSE
- ? R2
- ? Accuracy
- ? F1 Score
- ? Precision
- ? Recall

9.3 Fair Comparison Rules

Comparisons against external libraries must use:

- ? Same datasets.
- ? Same precision.
- ? Same thread counts.
- ? Same optimization settings.
- ? Same stopping criteria.
- ? Equivalent hyperparameters.

Single-threaded results should always be reported separately from multithreaded results.

10. Validation and Testing

10.1 Unit Testing

Every algorithm should include:

- ? Correctness tests.
- ? Edge-case tests.
- ? Shape validation.
- ? Numerical consistency checks.

10.2 Convergence Validation

Examples:

- ? Training loss decreases.
- ? Accuracy improves.

- ? Predictions stabilize.
- ? Expected synthetic behavior emerges.

10.3 Cross-Implementation Verification

Outputs should be compared against:

- ? Reference Python implementations.
- ? Analytical solutions where possible.
- ? Optional scikit-learn comparisons.

11. Proposed Repository Layout

```
ml-from-scratch/
|
|-- README.md
|-- docs/
|   |-- architecture/
|   |-- benchmarks/
|   |-- algorithms/
|   `-- optimization-notes/
|
|-- src/
|   |-- linear_models/
|   |-- tree_models/
|   |-- ensemble/
|   |-- svm/
|   |-- clustering/
|   |-- decomposition/
|   |-- preprocessing/
|   |-- metrics/
|   |-- model_selection/
|   `-- utils/
|
|-- csrc/
|   |-- linear_algebra/
|   |-- reductions/
|   |-- tree_kernels/
|   |-- distance_kernels/
|   `-- simd/
|
|-- scripts/
|   |-- build.sh
|   |-- benchmark.sh
|   `-- profile.sh
|
|-- benchmarks/
|-- datasets/
|-- experiments/
|-- tests/
`-- notebooks/
```


12. Long-Term Technical Goals

12.1 Medium-Term Goals

- ? Full classical ML coverage.
- ? Consistent estimator API.
- ? Benchmark suite.
- ? Cross-platform builds.
- ? Reusable optimization layer.

12.2 Advanced Future Extensions

Potential future directions:

- ? GPU kernels.
- ? Automatic differentiation.
- ? Computational graphs.
- ? Neural networks.
- ? Sparse tensor support.
- ? JIT compilation.
- ? Distributed training.
- ? Out-of-core learning.
- ? ONNX export.

These are exploratory extensions, not immediate objectives.

13. Educational Objectives

This project is intended to build expertise in:

Machine Learning:

- ? Supervised learning
- ? Unsupervised learning
- ? Optimization
- ? Ensemble methods
- ? Statistical learning

Mathematics:

- ? Linear algebra
- ? Calculus
- ? Probability
- ? Statistics
- ? Numerical methods

Systems Engineering:

- ? Memory layout
- ? CPU architecture
- ? SIMD
- ? Parallel programming
- ? Performance profiling

Software Engineering:

- ? API design
- ? Modular architecture
- ? Testing
- ? Benchmarking
- ? Reproducibility
- ? Documentation

14. What This Project Demonstrates

A successful implementation demonstrates:

- ? Deep understanding of classical machine learning.
- ? Ability to implement algorithms from first principles.
- ? Strong systems-level reasoning.
- ? Knowledge of numerical computing.
- ? Performance engineering capability.
- ? Cross-language integration skills.
- ? Software architecture maturity.

More importantly, it demonstrates the ability to move beyond simply using ML frameworks and toward understanding how they are built internally.

The project transforms machine learning from a black box into an engineered system.